TECHNICAL REPORT

ClaimGuard
Server Deployment and Test Report

What was installed, how it performs, what failed, and what to do next

Run `server_20261031` (about 2 hours, 31 October 2026)

Address https://claimguard.kernelsi.com

Server Ubuntu 26.04 in Proxmox, AMD Ryzen 7 5700X, 12 GiB RAM, NVIDIA RTX 3080 (10 GB)

Model google/gemma-3-4b-it (fp8) served by vLLM

Data Synthetic claims only. No real patient data was used.

Branch khalil_test, github.com/PublisherX02/Claim_Guard

Contents

1. Executive summary
2. What was deployed and how
3. How the tests work
4. Results, experiment by experiment (S1 to S7, S0)
5. Findings and recommendations
6. Problems met during the installation and how each was solved
7. Checklist before real use
8. What this test does not cover
9. Appendix: files, commands and glossary

1. Executive summary

ClaimGuard is installed and working on the server with every feature turned on: AI explanations of claim decisions, reading of claim documents (PDF, scan, photo) and search of the rulebook by meaning. It is reachable from anywhere at **https://claimguard.kernelsi.com** through Cloudflare. A test suite of eight experiments (S0 to S7) ran for about two hours. The script counted **20 of 26 measures as passed**. Two of the six failures come from how the test was run, not from the server, so the true position is 20 of 24 on the server's own behaviour.

376 / min
AI explanations (target 200)

11,721 / min
claims through the pipeline (target 200)

0 lost
claims lost in 5 restarts

2 ports
open on the server: 80 and 443

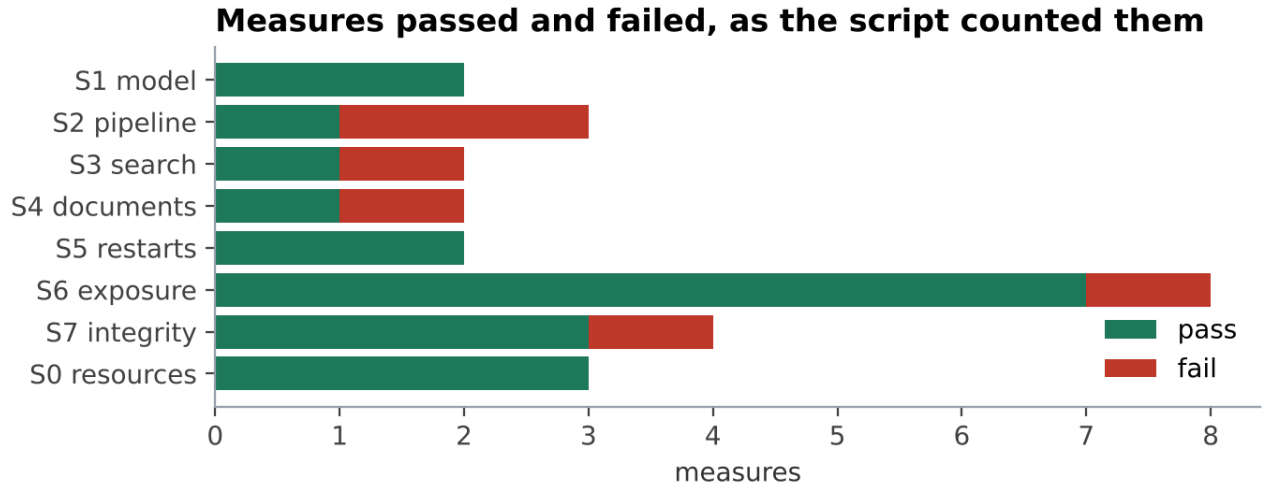


Figure 1. Measures passed and failed per experiment, as the test script counted them.

What works well

- **Speed.** The model and the claim pipeline run at nearly twice and about 58 times the target speed.
- **Safety of data.** Five services (worker, API, Redis, MongoDB, proxy) were restarted while claims were arriving. Every claim that the system had acknowledged was still there afterwards.
- **Security from the outside.** Only ports 80 and 443 listen on the server. Older encryption (TLS 1.0 and 1.1) is refused, security headers are present, and anonymous requests are refused with a 401.

- **Resources.** At the busiest moment the server used 75 % of its memory and 77 % of the GPU memory, so there is room to spare.

What needs work

Finding	Type	Importance
Phone photos are read correctly in only 50 % of fields (target 95 %)	Real	**High for document intake**
The API process peaked at 90 % of its 3 GiB memory limit	Real	**High, grows with data**
One batch of claims was rejected during the 30-minute steady load	Real, cause not yet known	**Medium**
One of five off-topic questions in search was answered instead of refused	Real, tunable	**Medium**
Port 8080 appeared open	Test artefact (Cloudflare answers on that port)	**None**
Health probe "signals" degraded	Test artefact (failed test logins)	**None, clears by itself**

2. What was deployed and how

2.1 The machine

Item	Value
Host	khalil, a Proxmox container running Ubuntu 26.04 LTS (kernel 7.0.14)
CPU	AMD Ryzen 7 5700X, 8 threads available
Memory	12 GiB (2 GiB swap)
GPU	NVIDIA GeForce RTX 3080, 10,240 MiB, driver 615.71.09
Disk	Enlarged from 60 GB to 99 GB during the install, because the images and models did not fit
Software	Docker 29.9.2, Docker Compose 5.6.1

2.2 The services

ClaimGuard runs as eight containers managed by Docker Compose. Each does one job:

Container	Role	Notes
proxy (Caddy)	Receives all web traffic, handles HTTPS	The only service that listens on the network (ports 80 and 443)
api	The ClaimGuard API and the web console	Written in Rust, memory limit 3 GiB
worker	Processes the queue of claims	Rust, memory limit 3 GiB
mongo	Database	MongoDB 7
redis	Queue and short-term storage	Redis 7
vllm	Writes the AI explanations on the GPU	Gemma 3 4B, fp8, 85 % of GPU memory reserved
embed	Search by meaning	bge-m3 model, runs on the CPU
reader	Reads documents by OCR	Tesseract

2.3 How a visitor reaches it

A browser opens `claimguard.kernelsi.com`. Cloudflare answers first, then forwards the request to the server, where Caddy passes it to the API. A Cloudflare rule sends anyone who types `http://` to `https://`. Inside the server, no other service is exposed.

2.4 Settings that must survive every update

Three files hold the changes that make the server work. A future update of the project must not overwrite them:

File	Why it matters
`.env`	Passwords, keys, model settings, the site name
`deploy/Caddyfile.internal`	Sends the real domain name to the API (`header_up Host`) and sets a default certificate name; without it the login fails with "That request came from another site"
`deploy/rust/compose.gpu.yml`	Gives the model server access to the GPU inside the container, mounts extra GPU libraries, sets memory limits

The earlier update followed this rule: the new files were copied in with those three excluded, after a backup of the database and the three files.

3. How the tests work

The suite lives in `experiments/server/` and was run as `run_all.py --run server_20261031`. Each experiment answers one question and compares the result with a target defined in advance (document 65 of the project).

Code	Question	Method
S1	How fast is the AI model?	Requests for explanations, 1 to 64 at the same time
S2	How fast is the whole claim pipeline?	Batches of 10 to 500 claims, then 30 minutes of steady load
S3	Does search by meaning find the right rule?	26 real questions and 5 off-topic strings
S4	Are documents read correctly?	40 test documents: digital PDF, image-only PDF, scan, photo
S5	Is anything lost when a service restarts?	Restart five services while claims arrive and count them
S6	What does the internet see?	Port scan, encryption test, headers, anonymous requests
S7	Is the audit trail intact?	Health panel, audit chain and ledger verification
S0	Are resources enough?	Memory, CPU and GPU sampled every ten seconds during the whole run (641 samples, 119 minutes)

One test was repeated. The first S5 run was not valid: it read claims back through a route that only serves the data loaded at start-up, not claims added later to the queue. The script was corrected to count the queue total before and after, with a preflight check that submits five claims and confirms the count rises by exactly five (it went from 6,870 to 6,875). Only the corrected result is reported here.

4. Results, experiment by experiment

S1. The AI model on the GPU: pass

Each request asks the model for one explanation (about 250 tokens in, up to 160 out). With one request at a time the model answers in 0.6 seconds. As more requests arrive together, throughput rises and levels off at about 377 explanations per minute from 8 simultaneous requests onwards. After that, extra requests only wait longer in line.

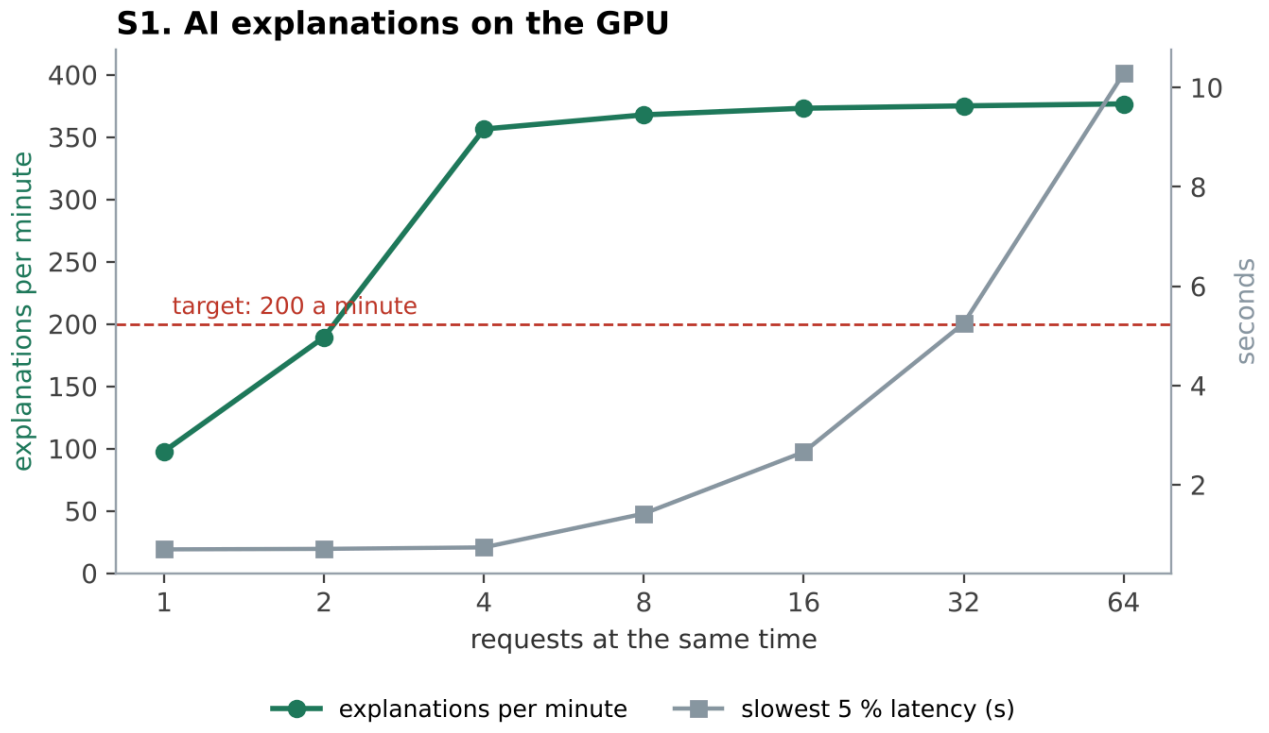


Figure 2. Throughput (green) levels off near 377 a minute; waiting time (grey) rises once the GPU is full.

At the same time	Requests	Errors	Per minute	Median (s)	Slowest 5 % (s)	Tokens/s
1	16	0	97.8	0.62	0.70	120
2	16	0	189.4	0.63	0.71	233
4	16	0	356.4	0.64	0.74	430
8	32	0	367.8	1.25	1.42	453
16	64	0	373.1	2.50	2.66	455
32	128	0	375.0	5.04	5.25	459
64	256	0	376.6	10.11	10.28	460

What it means. The target was 200 explanations a minute and there were no errors in 560 requests. The best operating point is 4 to 8 simultaneous requests: close to full speed, and an answer still arrives in about a second. The GPU is the limit, not the software.

S2. The claim pipeline: throughput passes, one batch failed

Claims travel through proxy, API, queue, engine and store. Small batches carry a fixed cost: a batch of 10 costs 26 ms per claim, while batches of 50 or more cost about 5 ms per claim. The best rate was

11,721 claims a minute with batches of 200.

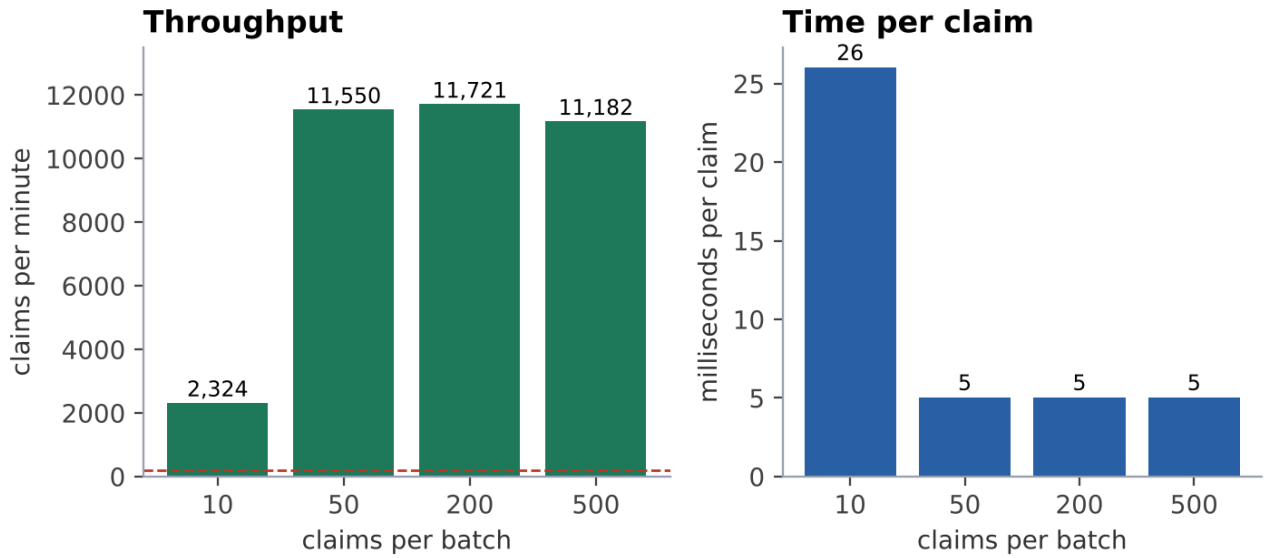


Figure 3. Claims per minute and milliseconds per claim, by batch size. Dashed line: target of 200 a minute.

Reading the system right after a heavy load

Right after 2,160 claims, the dashboard and the health panel answered in 0.7 to 1.2 seconds instead of milliseconds. They are still under the 2-second limit, but 8 health-panel requests were refused during that moment, which is why the script marks it as a fail.

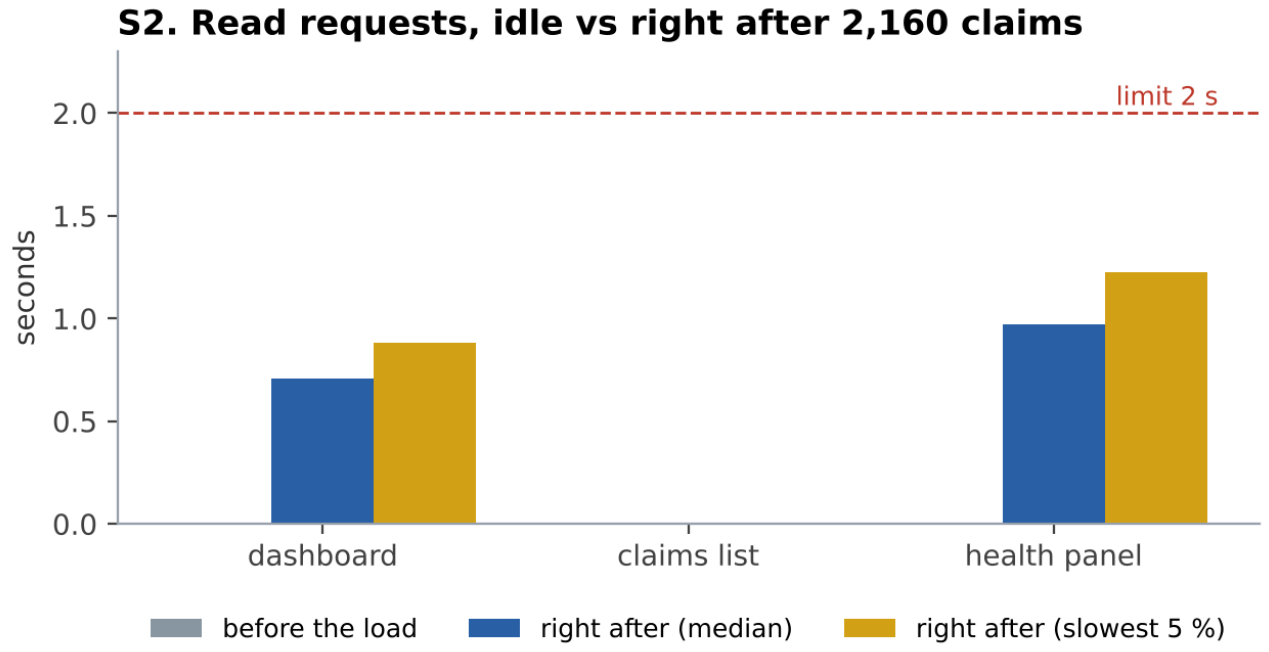


Figure 4. Answer time of three read requests, idle and right after the load.

Steady load for 30 minutes

2,990 claims were sent at a steady rate over 30 minutes. The batch time stayed flat at about 0.26 seconds. **One batch failed.** The cause is not recorded in the result files and has not been investigated yet. The API log from that period will show it.

Batch	Batches	Claims	Errors	Per minute	Batch done (s)	ms per claim
10	6	60	0	2,324	0.26	26
50	6	300	0	11,550	0.26	5
200	4	800	0	11,721	1.02	5
500	2	1,000	0	11,182	2.68	5

What it means. The pipeline is about 58 times faster than the target, so speed is not a concern. The open points are the single failed batch and the short slowdown of the read pages right after a very large burst.

S3. Search by meaning: quality passes, one off-topic string accepted

Staff type a question in their own words and the system finds the matching rule of the rulebook. **24 of 26 questions** returned the right rule (92 %, target 90 %). Speed is excellent: a median of 2 to 4 milliseconds, and up to 1,819 answers a second with 8 questions at the same time.

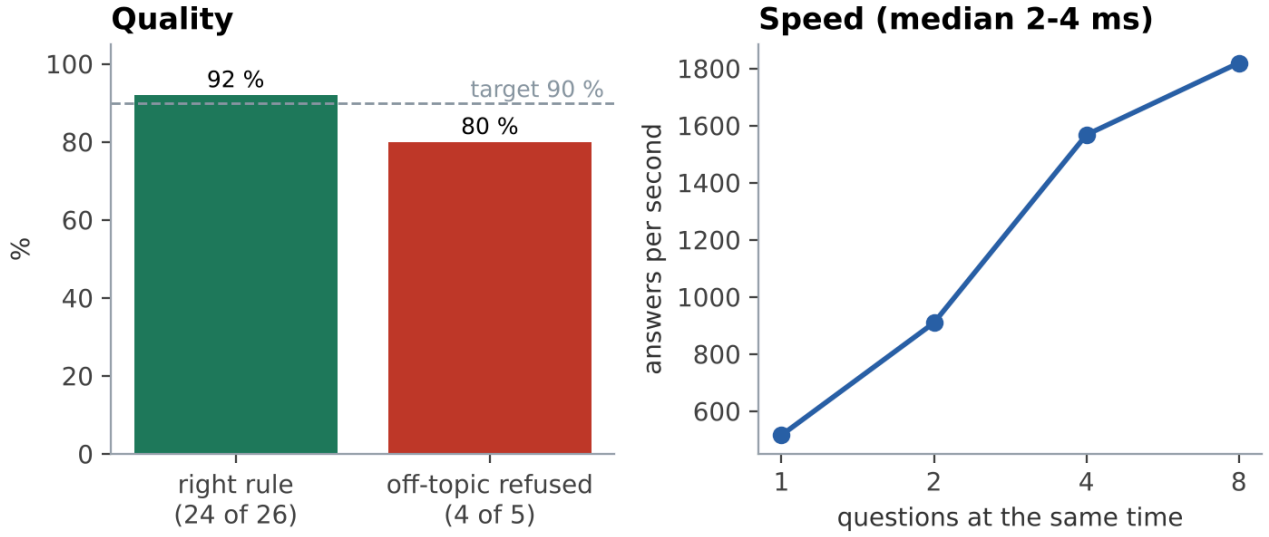


Figure 5. Left: quality against the 90 % target. Right: answers per second as questions are sent together.

- **Two questions missed:** "claim line without a service date" (expected rule R001) and "how many days after the service can a claim be submitted" (expected rule R014 and two policy tables).
- **One off-topic string was answered** instead of being refused (4 of 5 refused, target all). The refusal level is a setting (`EMBEDDER_THRESHOLD=0.485`). Raising it refuses more unrelated text but may also refuse a few genuine questions.
- No answer was blocked by the rate limit during the quality pass: all 26 requests returned 200.

S4. Documents to claims: fail on photos

40 documents were read, 0 failed to read. Accuracy depends strongly on the type of document. Only the Tesseract OCR reader was used; no vision model was installed.

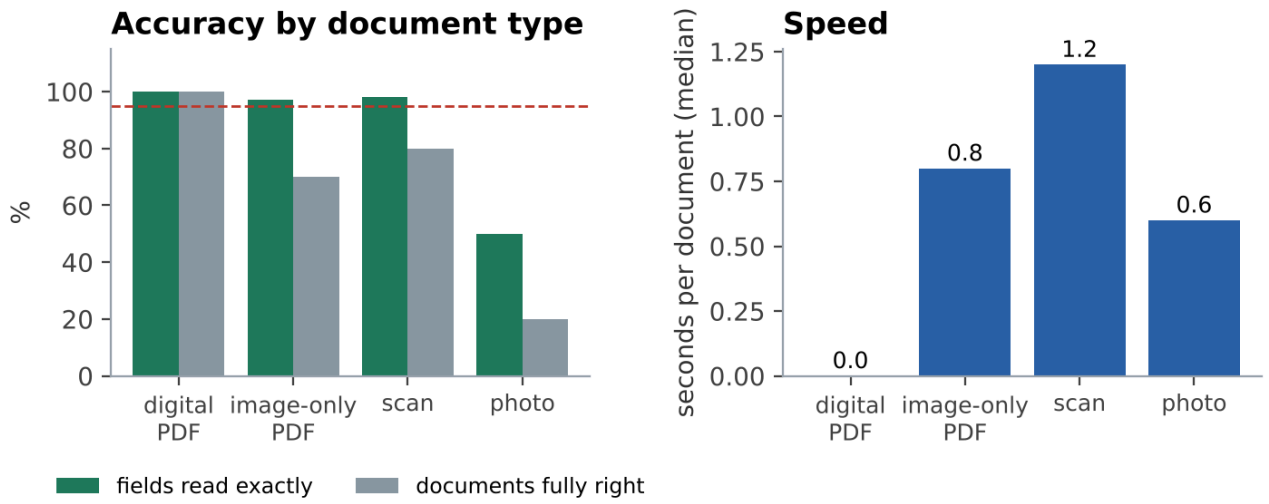


Figure 6. Left: share of fields read exactly (green) and documents with every field right (grey). Red line: 95 % target. Right: seconds per document.

Document type	Documents	Fields exact	Fully right	Median time (s)
Digital PDF	10	100.0 %	100 %	0.0
Image-only PDF	10	97.3 %	70 %	0.8
Scan	10	98.2 %	80 %	1.2
Photo	10	50.0 %	20 %	0.6
All	40	**86.4 %**	67.5 %	

What it means. Digital PDFs, scans and image-only PDFs reach 97 to 100 % of fields. Phone photos reach only 50 %, which pulls the overall result to 86.4 % against a 95 % target. The earlier project measurement was 97.4 % of fields with two readers on the build PC. Until a vision model is added (the `vision` profile), ask staff to submit scans or PDFs, not phone photos, and keep a human check on every document read.

S5. Restarts under load: pass

Each service was restarted while a background client kept sending claims in batches of five. After each restart, the queue total was compared with the number of claims the system had acknowledged. A claim that was acknowledged and then missing would be the one failure that matters.

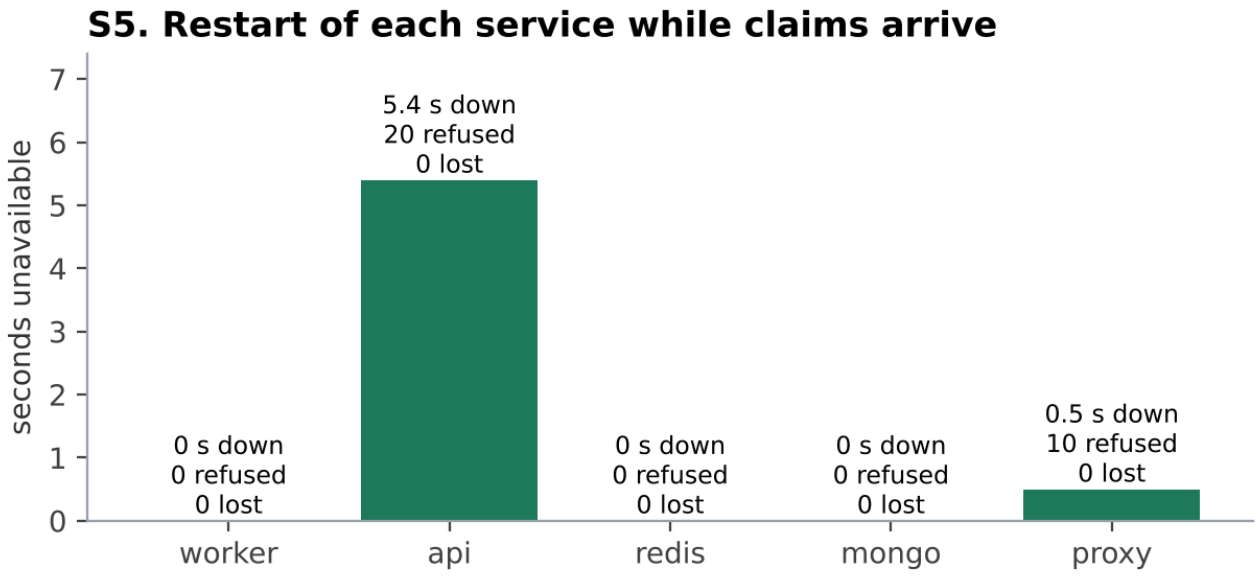


Figure 7. Seconds the service was unavailable, requests refused, and claims lost, per restarted service.

Service	Restart (s)	Unavailable (s)	Back	Sent	Acknowledged	Refused	Lost
worker	4.3	0.0	yes	430	430	0	**0**
api	5.6	5.4	yes	320	300	20	**0**
redis	0.3	0.0	yes	380	380	0	**0**
mongo	0.5	0.0	yes	360	360	0	**0**
proxy	0.4	0.5	yes	205	195	10	**0**

What it means. Nothing acknowledged was lost, and every service came back. Restarting the API makes the site unavailable for about 5 seconds, and the proxy for half a second. Requests sent in that window are refused, not silently dropped, so a client that retries loses nothing. Restarts of the worker, Redis and MongoDB were invisible to users. Plan updates for a quiet moment, since the API restart is a real, if short, interruption.

S6. Exposure seen from the network: pass in practice

This test looks at the server like an attacker would, from outside. It ran against the domain, so Cloudflare sits between the test and the server.

Check	Result
Database, Redis, model server, embedding, reader, Ollama, app and SSH ports	**All closed**
HTTPS open, TLS 1.3 negotiated	**Yes**
TLS 1.0 and 1.1	**Refused**
HTTP on port 80	**Redirects to HTTPS (301)**
Security headers (HSTS, content-type protection, content security policy, framing protection, no server version)	**All present**
Anonymous request to the API and the health panel	**401, refused**
`/healthz` shows only a minimal answer	**Yes**
Port 8080 "open"	**Test artefact: Cloudflare answers on 8080**

> **Two things happened here.** First, the port 8080 reading: on the server itself, `ss -tlnp` shows only Caddy listening, on ports 80 and 443. Nothing listens on 8080, so the reading is Cloudflare's own port. Second, the redirect check failed twice for real reasons: the first time the redirect was relative (`/ui/`) and the visitor stayed on plain HTTP; the second time the Cloudflare rule had a typo (`ttps://`). Both were fixed, and the final run passes. The certificate seen from outside belongs to Cloudflare (Google Trust Services), with 83 days left.

S7. Integrity and audit: pass, one optional probe degraded

The audit chain and the ledger both verify, and 13 of 15 health probes are "ok". `signals`, which counts failed sign-in attempts, showed "degraded" because of the failed logins made while preparing the test accounts; it returns to normal on its own after about an hour without failed sign-ins. The probe `timestamp` reads "unknown", most likely because no external timestamp service is set up (not checked).

S0. Resources during the run: pass, with one warning

Memory, CPU and GPU were sampled throughout the 119 minutes (641 samples). The server never had less than 3.1 GiB of its 12 GiB available. The GPU peaked at 7,839 MiB of 10,240 MiB (77 %), 100 % utilisation, 68 °C and 336 W during the model test.

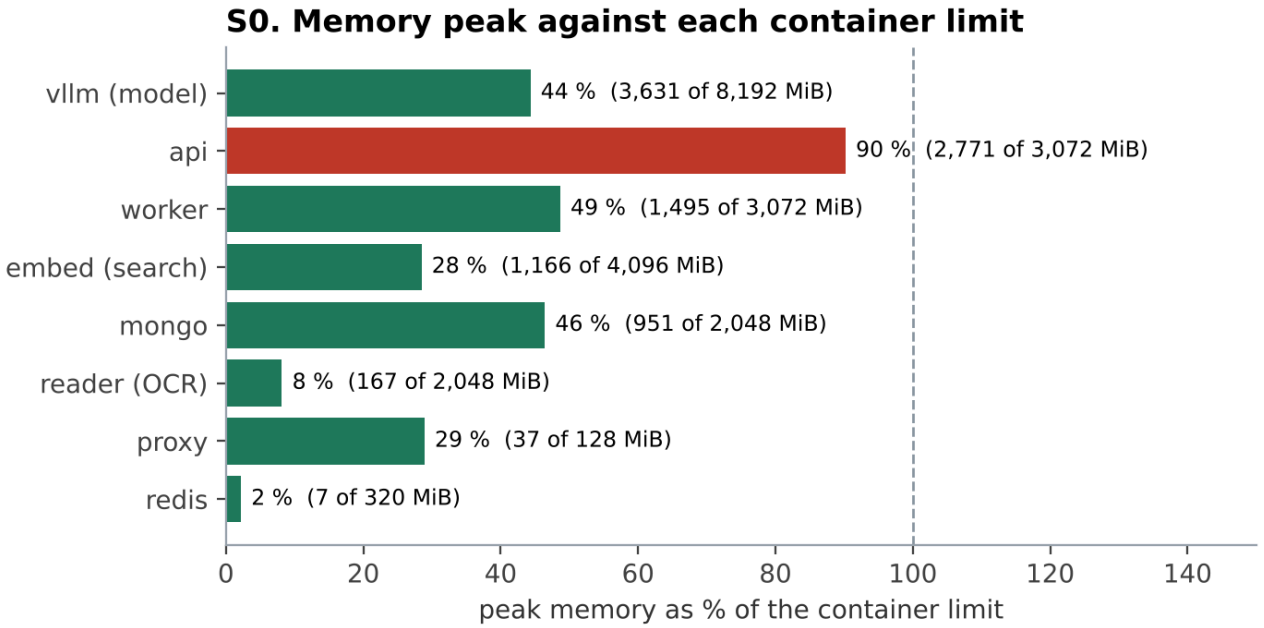


Figure 8. Peak memory of each container as a share of its limit.

Container	Peak (MiB)	Limit (MiB)	Share
vllm	3,631	8,192	44 %
api	2,771	3,072	**90 %**
worker	1,495	3,072	49 %
embed	1,166	4,096	28 %
mongo	951	2,048	46 %
reader	167	2,048	8 %
proxy	37	128	29 %
redis	7	320	2 %

The API is the one to watch. It originally had a 768 MiB limit and was killed by the system (exit code 137) when about 7,000 test claims had accumulated; the site then returned 502 errors through Cloudflare. The limit was raised to 3 GiB for the API and the worker. Even so, the API reached 2.77 GiB with about 8,500 claims in the queue, because it keeps the claims in memory. With real data, memory use will grow with the number of claims. Starting from a clean database and watching the API memory is the safe approach.

5. Findings and recommendations

#	Finding	Cause	Recommended action	Priority
1	Photos read at 50 % (S4)	OCR (Tesseract) only; phone photos have glare, tilt and blur	Add the `vision` profile and a vision model; until then accept scans and PDFs only and keep human review. Re-run S4 afterwards	**High**
2	API at 90 % of memory (S0)	Claims kept in memory; 8,500 test claims	Start with a clean database; keep the 3 GiB limit; check `docker stats` weekly; raise the limit if it passes 80 %	**High**
3	One failed batch in the steady load (S2)	Not recorded	Read the API log for that period; repeat the 30-minute soak after clean-up and record the error code	**Medium**
4	One off-topic string answered (S3)	Refusal threshold too lenient for that string	Test `EMBEDDER_THRESHOLD` slightly higher (for example 0.50) against the 26 questions and the 5 off-topic strings, and keep the best balance	**Medium**
5	API restart interrupts for about 5 s (S5)	Normal for one API instance	Apply updates in quiet hours; clients should retry refused requests	**Low**
6	Redirect from HTTP depends on a Cloudflare rule	Server cannot see the visitor's original scheme	Turn on "Always Use HTTPS" in Cloudflare if the domain owner agrees; keep the rule meanwhile	**Low**

6. Problems met during the installation

These are recorded so the next installation, or a rebuild after a failure, goes faster.

Problem	Cause	Solution
`curl` not found	Minimal system	Install curl and other basic tools with apt
GPU not usable in Docker (BPF error)	Proxmox container blocks cgroup control	Run the NVIDIA toolkit in no-cgroups mode
GPU libraries missing in containers	Versioned library files absent	Copy the `.so.1` files to versioned names; use CDI device access
Disk full while pulling images	60 GB disk too small	Enlarge to 99 GB; prune the build cache
Model download refused (403)	Gemma is a gated model; token empty	Accept the licence on Hugging Face; set the token silently (`read -rs`); revoke a token that had been exposed
vLLM would not start (missing CUDA 13 libraries)	Libraries not in the image	Mount `/opt/nvrtc13` and set `LD_LIBRARY_PATH`
vLLM compile error (noexec)	`/tmp` mounted without exec	`tmpfs /tmp:size=4g,exec`
vLLM: no memory for the KV cache	Too little GPU memory reserved	`VLLM_GPU_UTIL=0.85`
Pasting broke the YAML	PuTTY removes leading spaces on paste	Write files with `printf` and `sed`
Browser error SSL protocol	No default certificate name for access by IP	`default_sni` in Caddy
"That request came from another site"	The API compares the Origin header with the Host header; the proxy sent the internal name	`header_up Host claimguard.kernelsi.com` in each proxy block
Search service would not start (offline error)	Model name not resolvable offline	Point `EMBED_MODEL` to the local model folder
Test scripts blocked by Cloudflare (error 1010)	Python user agent refused	Run the scripts against the server's IP address
Weak password refused	Policy: 12 characters or more, three character classes	Use a stronger password
Login failed for one test account	Special characters pasted wrongly	Create a new account with a simpler password
Report script crashed (division by zero)	A container with no memory limit	Guard at line 315 of `make_server_report.py`. This fix is on the server only and is lost on update unless re-applied
Site returned 502; API exit code 137	Memory limit 768 MiB too low for thousands of claims	Raise the API and worker limits to 3 GiB

7. Checklist before real use

1. **Remove the temporary rate limit.** Delete `ASK_PER_MINUTE=100000` from `.env` and recreate the API container.
2. **Start clean.** Either a fresh database (removes about 8,500 test claims and all accounts; create the real admin with `cg admin create-admin`) or keep the data and delete the benchmark accounts CG-4002, CG-4003 and CG-1001 after a real administrator exists.
3. **Protect the backups.** `backup-before-update/` contains the encryption key; store it, with the `.env` copy, off the Downloads folder (encrypted drive or password manager). Never commit them to GitHub.
4. **Delete the Hugging Face token** that is no longer needed, and keep SSH (port 22) closed to the internet.
5. **Delete temporary files** on the server: `~/bench.env`, `~/claimguard-new.zip`, `~/update-tmp`.
6. **Re-run later:** S7 after an hour without failed sign-ins; S4 after a vision model is added; S2 soak after clean-up.
7. **Schedule a backup** of the database and a restore test.

8. What this test does not cover

- All claims and documents were **synthetic**. Real hospital paper, handwriting and Arabic documents are untested.
- Results are for **one server** and one user at a time on the console. Many reviewers working together on the web pages were not simulated.
- Accuracy of the AI explanations (are they correct and useful to a reviewer?) was not scored. S1 measures speed only.
- Failure of the whole machine, loss of the disk and restoration from backup were not tested.
- Security testing here is a view from the network, not a penetration test.

9. Appendix

9.1 Files in the repository (branch khalil_test)

File	Content
`docs/66_Server_Experiment_Results.md`	Results as written by the script
`docs/figures/server/`	Charts of the run
`docs/reports/ClaimGuard_Server_Report.pdf`	This report
`experiments/server/s05_resilience.py`	Corrected restart test
`outputs/server/server_20261031/`	Raw results (report.md, s06_exposure.json)

9.2 Commands used

Purpose	Command
Run the experiments (inside tmux, in the virtual environment)	`python experiments/server/run_all.py --run server_20261031 --docs ~/docs_test --soak-minutes 30 --yes-restart`
Run only the restart test	`python experiments/server/run_all.py --run server_20261031 --only s05 --yes-restart`
Exposure test (from another machine)	`python experiments\server\s06_exposure.py --run server_20261031 --host claimguard.kernelsi.com`
Rebuild the report	`python experiments/server/make_server_report.py --run server_20261031`
Pack the results	`python experiments/server/pack_results.py --run server_20261031`
Listening ports on the server	`sudo ss -tlnp`

9.3 Glossary

Term	Meaning
Slowest 5 % (p95)	95 out of 100 requests were at least this fast
Median (p50)	The typical request: half were faster, half slower
OCR	Optical character recognition: reading text from an image
vLLM	The program that runs the AI model on the GPU
TLS	The encryption behind HTTPS; versions 1.0 and 1.1 are obsolete
HSTS	A header that tells browsers to always use HTTPS
Soak test	A steady load over a long time to reveal slow leaks and rare failures
Exit code 137	The container was killed, usually for exceeding its memory limit